

RaptorRuntime Product and Architecture Specification

Original standalone JavaScript and TypeScript runtime for the Raptor ecosystem

RaptorRuntime is the proposed native execution platform beneath RaptorJS. It will be installed independently, run JavaScript and TypeScript applications without requiring Node.js at runtime, and provide a coherent foundation for the existing Raptor packages. This document defines the product boundary, architecture, compatibility policy, delivery plan, and decisions needed to build it without copying Node.js source code, branding, or internal design.

Decision	Specification
Product name	RaptorRuntime
Primary command	raptor
Initial delivery	Standalone native executables and signed installers
Primary languages	JavaScript and TypeScript
Relationship to RaptorJS	Runtime layer below the existing framework packages
Originality rule	Independent implementation and API design; no copied Node.js code or branding

1 Product Intent

The goal is not to recreate Node.js feature by feature. RaptorRuntime should solve the same broad problem - executing server-side and tooling-oriented JavaScript - with an original operating model: explicit capabilities, built-in observability, deterministic task execution, and a small stable platform API. Developers should be able to install one product and immediately create, run, test, package, and inspect a Raptor application.

- Make TypeScript a first-class source format rather than a separate compile step.
- Provide one integrated command surface instead of a collection of unrelated global tools.
- Keep application permissions visible and enforceable from development through deployment.
- Support the RaptorJS packages as first-party consumers without coupling the framework to runtime internals.
- Offer a pragmatic migration bridge for npm packages while preserving a distinct Raptor-native API.

2 Product Boundary and Naming

RaptorRuntime owns process startup, language execution, module loading, task scheduling, operating-system access, networking primitives, package resolution, diagnostics, and distribution. RaptorJS remains an application and UI framework. The existing `@raptor/run` package keeps its present framework role; it is not renamed or replaced by the runtime during the first phases.

Layer	Name	Responsibility	Collision policy
Runtime	RaptorRuntime / raptor	Execution platform and developer CLI	Owns raptor runtime subcommands
Framework	@raptor/run	Application runtime, SSR and routing	Keeps current package and public imports
Build	@raptor/bundle	Framework bundling and graph tooling	May use runtime APIs but stays independently versioned

Layer	Name	Responsibility	Collision policy
Engine	@raptor/engine	Raptor application planning and code generation	Existing raptor command remains until command migration is explicitly approved

During development the native binary is called raptor-runtime to avoid overwriting the current raptor executable. The final unified raptor command is introduced only after a compatibility audit. Early commands are raptor-runtime run, raptor-runtime test, raptor-runtime doctor, and raptor-runtime pack.

3 Originality and Intellectual Property Guardrails

RaptorRuntime may interoperate with public web, ECMAScript, TypeScript, POSIX, Windows, npm registry, and package-format specifications. Interoperability is not a license to copy an implementation. The project must use clean-room engineering practices: design from published standards and independently written tests; review dependencies and licenses; document provenance for imported specifications; and reject copied source, copied internal test suites, copied error text, or Node branding.

Allowed foundation	Not allowed	Required practice
ECMAScript and Web Platform standards	Node.js source code or internal architecture	Link every API decision to a public standard or Raptor design record
V8, QuickJS, JavaScriptCore under their licenses	Node trademarks or product naming	Maintain third-party notices and license review
npm registry metadata and published package contracts	Copying package-manager implementation code	Build an independent resolver and lockfile format
Publicly documented interoperability behavior	Node-specific undocumented quirks as compatibility targets	Treat compatibility as measured behavior, not imitation

4 User Experience

A new installation must work without Node.js, a global package manager, or a compiler configuration. The runtime detects a project manifest, applies a clear default, and explains any requested capability before granting it. Every command supports human-readable output and structured JSON output for automation.

Command	Purpose	First release behavior
raptor-runtime run app.ts	Run an application	Transpile TypeScript in memory, load modules, apply permissions
raptor-runtime test	Run tests	Discover *.test.ts and execute with isolated clocks and structured reporting
raptor-runtime init	Create project	Generate a minimal manifest, source file and permissions file
raptor-runtime pack	Build deployable unit	Emit reproducible application bundle with lockfile
raptor-runtime doctor	Diagnose environment	Report binary version, target platform, project state and capability issues
raptor-runtime trace	Inspect execution	Write OpenTelemetry-compatible trace data and request timings

5 System Architecture

The runtime is a native host with a small JavaScript binding surface. Rust is the recommended systems language because it supports safe concurrency, portable builds, strong tooling, and an explicit dependency model. A supported JavaScript engine is embedded behind an internal engine adapter so a future engine replacement does not change application APIs.

Component	Responsibilities	Public exposure
Launcher	Parse CLI, locate project, start diagnostics	raptor-runtime executable
Engine adapter	Initialize isolate, evaluate modules, map errors	None
Module graph	Resolve Raptor modules, local files, npm bridge packages and URL modules if enabled	raptor:modules diagnostics only
Capability broker	Approve filesystem, network, process, clock and secret access	raptor:capabilities
Task fabric	Schedule async work, cancellation, workers and resource quotas	raptor:tasks
Platform services	HTTP, file storage, crypto, subprocesses, streams	raptor:* modules
Telemetry core	Logs, metrics, traces and profiles	raptor:observe

Internal layers communicate through typed Rust interfaces and versioned message contracts. JavaScript never receives raw host handles. Each host object is scoped to an isolate and a granted capability, which prevents accidental reuse across workers or tenants.

6 Raptor Native API Design

Raptor APIs use the raptor: module namespace. This avoids pretending to be a built-in Node module and keeps the product vocabulary stable. APIs are promise-based, use Web-standard Request, Response, URL, Headers, ReadableStream and AbortSignal types where appropriate, and expose explicit options instead of ambient global state.

Module	Initial scope	Design rule
raptor:serve	HTTP server, routes, graceful shutdown	Fetch-native request and response contract
raptor:files	Scoped files, atomic writes, directory traversal	Requires an explicit filesystem capability
raptor:net	Outbound fetch, TCP and TLS where enabled	Destination allowlists and timeouts are first-class
raptor:tasks	Workers, queues, cancellation and deadlines	No shared mutable host state
raptor:kv	Local development key value store interface	Portable interface, replaceable backing store
raptor:observe	Logger, spans, metrics and profiles	Structured events from the start
raptor:process	Arguments, environment and controlled child processes	No unrestricted host access by default

7 Security Model

Security is a product feature, not an optional wrapper. A project declares what it needs in raptor.runtime.json. The default development policy prompts or denies sensitive access; production policies are non-interactive and auditable. Capabilities are granular, revocable, inherited only when explicitly delegated, and recorded in diagnostics.

Capability	Example declaration	Default
Filesystem read	files.read: [./src, ./assets]	Denied outside project
Filesystem write	files.write: [./raptor, ./dist]	Denied

Capability	Example declaration	Default
Network	net.connect: [api.example.com:443]	Denied
Environment	env.read: [DATABASE_URL]	Denied
Subprocess	process.spawn: [git]	Denied
Time and randomness	clock.real and crypto.random	Allowed with trace annotations

8 Package and Compatibility Strategy

RaptorRuntime uses a Raptor manifest and lockfile, not a cloned package-manager design. The runtime can fetch packages from npm-compatible registries through an adapter, but Raptor-native projects declare dependencies, integrity data, target platforms and permissions in their own schema. The registry protocol remains replaceable.

- Phase one supports standard ESM packages with a defined subset of package metadata.
- CommonJS runs only through a contained compatibility adapter and is never the default execution model.
- Packages that require Node internal bindings or undocumented behavior are reported as unsupported rather than emulated indefinitely.
- Raptor packages publish normal semver metadata and declare the minimum RaptorRuntime version they require.
- The lockfile stores content integrity and origin, enabling reproducible builds and supply-chain audits.

9 Integration with the Existing Repository

The repository already contains framework and tooling packages such as `@raptor/core`, `@raptor/dom`, `@raptor/server`, `@raptor/run`, `@raptor/bundle`, `@raptor/engine`, and the RaptorWire packages. RaptorRuntime must initially be additive. No current command, package name, or import path should change merely because the runtime work starts.

Rule	Implementation
Separate workspace	Create packages/runtime for host contracts and runtime-cli for the development launcher; native host may live in runtime-native
No command overwrite	Use raptor-runtime while existing raptor, raptor-run and raptor-bundle binaries remain registered
No forced dependency	Existing TypeScript packages continue to run under the currently declared Node engine during bootstrap
Adapter boundary	Add <code>@raptor/runtime-bridge</code> only after the native APIs are stable and covered by contract tests
Independent versions	Runtime release cadence and framework release cadence are linked by compatibility ranges, not a monorepo-wide lockstep

10 Distribution and Installation

RaptorRuntime is distributed as a signed, versioned native product. The installer places the executable on PATH and creates an update channel. It does not bundle or depend on a Node.js installation. Development builds are labelled clearly and use a distinct command name until release validation completes.

Platform	Delivery	Verification
Windows	MSIX or signed installer, plus winget manifest	Authenticode signature, SHA 256 checksum, clean-machine install test
macOS	Signed and notarized PKG or DMG, plus Homebrew formula	Notarization, universal binary test, checksum
Linux	Tarball, deb and rpm packages, plus package-manager metadata	GPG or Sigstore provenance, container smoke test

Platform	Delivery	Verification
CI and containers	Pinned release archives and minimal base images	Digest-pinned artifact and reproducible smoke application

11 Delivery Roadmap

Milestone	Outcome	Exit criteria
0 Architecture spike	Native binary evaluates a local JavaScript file	Supported engine runs a module; startup, error and shutdown paths tested
1 Developer core	TypeScript, ESM graph, permissions, run and doctor commands	Can start a Raptor example without Node at runtime
2 Services	Fetch server, files, networking, workers and telemetry	Load and error tests; capabilities enforced in integration tests
3 Packaging	Manifest, lockfile, registry adapter and pack command	Reproducible application bundle on Windows, macOS and Linux
4 Framework bridge	RaptorJS runs on supported native APIs	Selected existing examples pass through the bridge
5 Public preview	Installers, docs, migration guide and issue triage	Security review, license audit, performance baselines and signed releases

12 Quality Gates

- Every public API has a written contract, examples, capability behavior and a negative test.
- Compatibility claims are backed by a published, versioned test matrix rather than a blanket promise.
- A clean-room provenance review is required for new platform APIs and compatibility adapters.
- Native dependencies receive license, vulnerability and update review before release.
- Release candidates pass installation, upgrade, uninstall, permission-denial, network-failure and crash-recovery tests on all supported platforms.
- Performance reports track cold start, HTTP throughput, memory ceiling and task-cancellation latency against RaptorRuntime's own prior releases.

13 Decisions Required Before Implementation

The next implementation phase needs four explicit product decisions. The recommendations below optimize for a small original core and a safe bridge to the existing repository.

Decision	Recommendation	Reason
JavaScript engine	Begin with V8 through a narrow adapter	Mature performance and tooling; adapter preserves future choice
Native language	Rust	Strong safety model, cross-platform builds and modern async ecosystem
First target	Developer server applications	Best fit with existing @raptor/server and @raptor/run work
Compatibility promise	ESM first, selective npm bridge	Useful adoption path without recreating every legacy behavior
Command migration	Keep raptor-runtime until audit passes	Prevents collisions with existing repository binaries

14 Immediate Next Work

The first engineering task is a short architecture spike, not a production runtime. It should prove that a standalone executable can load one TypeScript module, print structured diagnostics, enforce a denied filesystem access, and shut down cleanly. The spike should live alongside - not inside - the existing `@raptor/run` package. Its acceptance tests become the first RaptorRuntime contract suite.

First Spike Acceptance Checklist

Check	Evidence required
Standalone launch	A native development binary starts without Node.js installed on the machine.
TypeScript module	One local TypeScript entry module evaluates and reports its result through structured diagnostics.
Permission denial	An attempted read outside the declared project scope fails with a clear capability error.
Clean shutdown	The process closes workers and handles cleanly after success, failure and cancellation.
Contract tests	The five behaviors above are covered by repeatable automated tests and become the baseline suite.

The spike is successful only when all checks pass on a clean Windows environment. It remains an isolated experiment until the architecture, licenses, security boundary and command ownership are approved for the first production milestone.